

```
In [5]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns

url = 'https://raw.githubusercontent.com/selva86/datasets/master/BostonHousing.csv'
df = pd.read_csv(url)

print(df.head())
print(df.isnull().sum())

X= df.drop('medv', axis = 1)
y= df['medv']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)
y_pred = lr_model.predict(X_test_scaled)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error: {mse}')
print(f'Root Mean Squared Error: {rmse}')
print(f'R-squared: {r2}')

plt.figure(figsize=(10,6))
sns.regplot(x=y_test, y=y_pred, scatter_kws={"s": 60}, line_kws={"color": "red"})
plt.xlabel("Actual Prices")
```

```
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted House Prices")
plt.show()
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	\
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	

	b	lstat	medv
0	396.90	4.98	24.0
1	396.90	9.14	21.6
2	392.83	4.03	34.7
3	394.63	2.94	33.4
4	396.90	5.33	36.2

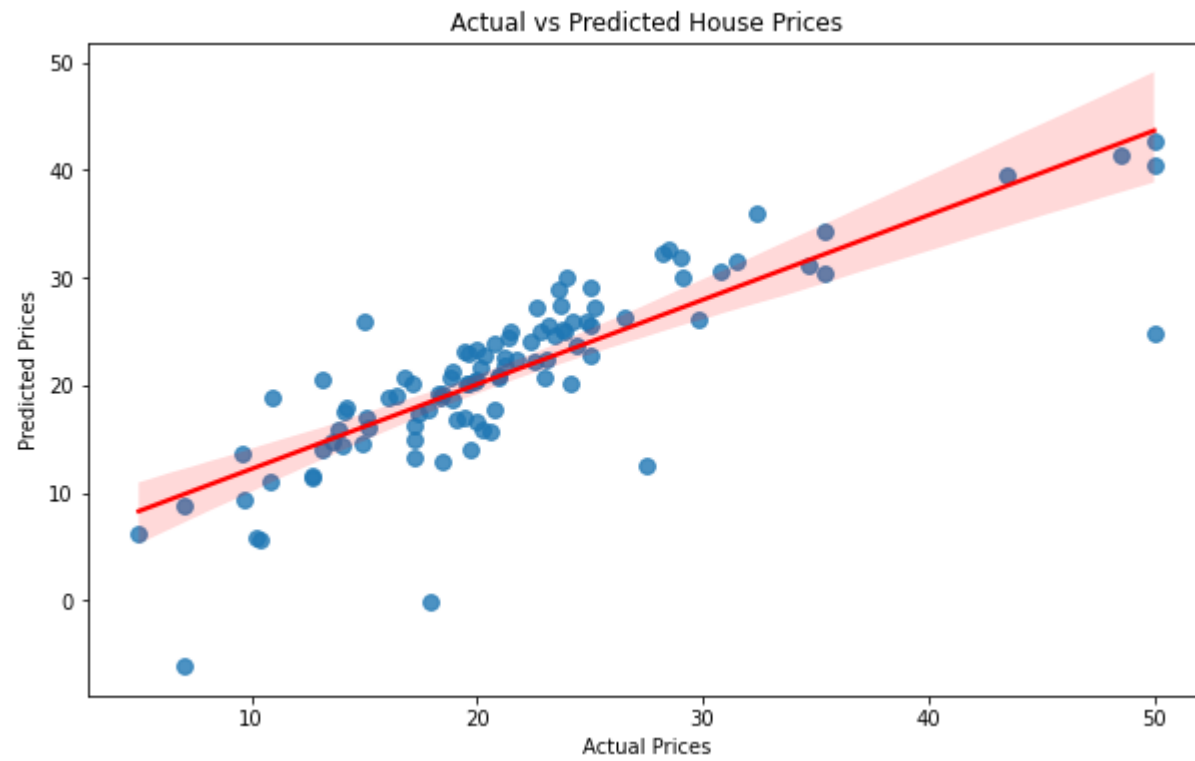
crim 0
 zn 0
 indus 0
 chas 0
 nox 0
 rm 0
 age 0
 dis 0
 rad 0
 tax 0
 ptratio 0
 b 0
 lstat 0
 medv 0

dtype: int64

Mean Squared Error: 24.29111947497352

Root Mean Squared Error: 4.928602182665337

R-squared: 0.668759493535632



```
In [10]: selected_features = ['rm', 'lstat', 'ptratio', 'indus', 'tax']

metrics = {
    'Feature': [],
    'MSE': [],
    'RMSE': [],
    'R-squared': []
}

def analyzed_feature(feature):
    X = df[[feature]]
    y = df['medv']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)
    lr_model = LinearRegression()
    lr_model.fit(X_train_scaled, y_train)
    y_pred = lr_model.predict(X_test_scaled)

    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    metrics['Feature'].append(feature)
    metrics['MSE'].append(mse)
    metrics['RMSE'].append(rmse)
    metrics['R-squared'].append(r2)
    plt.figure(figsize=(10, 6))
    sns.regplot(x=y_test, y=y_pred, scatter_kws={"s": 60}, line_kws={"color": "red"})
    plt.xlabel("Actual Prices")
    plt.ylabel("Predicted Prices")
    plt.title(f"Actual vs Predicted House Prices ({feature})")
    plt.show()

for feature in selected_features:
    analyzed_feature(feature)

metrics_df = pd.DataFrame(metrics)
```

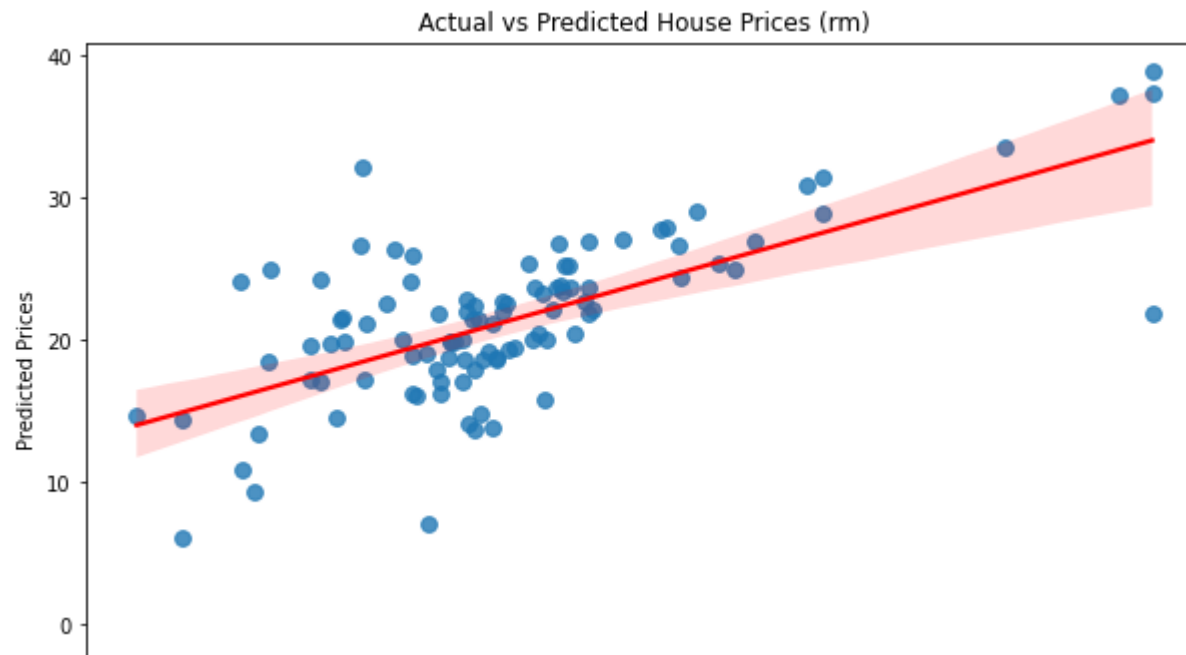
```
fig, ax = plt.subplots(1, 3, figsize=(18, 5))

# MSE Comparison
sns.barplot(x='Feature', y='MSE', data=metrics_df, ax=ax[0])
ax[0].set_title('MSE Comparison')
ax[0].set_xticklabels(metrics_df['Feature'], rotation=45)

# RMSE Comparison
sns.barplot(x='Feature', y='RMSE', data=metrics_df, ax=ax[1])
ax[1].set_title('RMSE Comparison')
ax[1].set_xticklabels(metrics_df['Feature'], rotation=45)

# R-squared Comparison
sns.barplot(x='Feature', y='R-squared', data=metrics_df, ax=ax[2])
ax[2].set_title('R-squared Comparison')
ax[2].set_xticklabels(metrics_df['Feature'], rotation=45)

plt.tight_layout()
plt.show()
```



```
In [9]: selected_features = ['rm', 'lstat', 'ptratio', 'indus', 'tax']

def multiple_linear_regression(features):
    X = df[features]
    y = df['medv']

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    lr_model = LinearRegression()

    lr_model.fit(X_train_scaled, y_train)

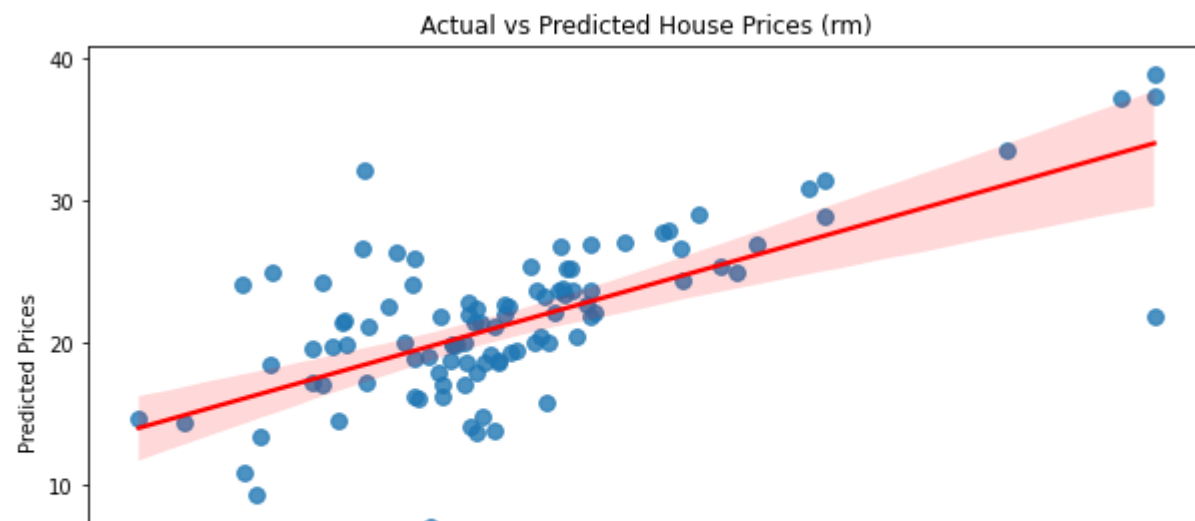
    y_pred = lr_model.predict(X_test_scaled)

    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    print(f"Multiple Linear Regression Results with features: {features}")
    print(f"Mean Squared Error: {mse}")
    print(f"Root Mean Squared Error: {rmse}")
    print(f"R-squared: {r2}")

    plt.figure(figsize=(10, 6))
    sns.regplot(x=y_test, y=y_pred, scatter_kws={"s": 60}, line_kws={"color": "red"})
    plt.xlabel("Actual Prices")
    plt.ylabel("Predicted Prices")
    plt.title("Actual vs Predicted House Prices (Multiple Linear Regression)")
    plt.show()

multiple_linear_regression(selected_features)
```



In []: